



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

18 · Guía del nuevo desarrollador

Documentación de sistema · Empresa Operativa Chile

Documento 18 de la serie

Versión del manifiesto 1.5.0 · commit 25f1a5e

Análisis del 2026-09-22

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

18 · Guía del nuevo desarrollador

Fuente de verdad del proceso de contribución: `CONTRIBUTING.md` . Esta

guía no lo repite: añade el **itinerario de lectura** y el mapa de dónde tocar cada cosa.

Lo primero, en cinco minutos

```
git clone https://github.com/vladimiracunadev-create/empresa-operativa-chile.git
cd empresa-operativa-chile
node scripts/build-all.mjs
node apps/empresa-operativa/server.mjs
```

Abre `http://127.0.0.1:4180` , pulsa **SANDBOX** y trastea. Hay una empresa de ejemplo con seis meses de operaciones.

No hace falta `pnpm install` . El proyecto no tiene dependencias de producción ni de desarrollo: sólo Node ≥ 20 . `pnpm install` sólo se necesita para empaquetar Android o Windows.

Comprueba que todo está sano:

```
node --test tests/*.test.mjs
```

Deberías ver `pass 153` , en menos de un segundo.

Las cinco cosas que hay que saber antes de tocar nada

Si sólo lees una sección de esta guía, que sea ésta.

1. `pnpm`, nunca `npm`

Está declarado en `package.json` como `packageManager: pnpm@11.2.2`. `pnpm-lock.yaml` es el lockfile válido y `npm install` lo ignoraría.

2. Nada que viaje al dispositivo puede importar `node:*`

Es la regla dura de la arquitectura. Si un archivo que acaba en `apps/web/dist` importa `node:fs`, `node:path` o cualquier otro, la app queda en blanco en el APK sin un error visible.

`scripts/build-web.mjs` aborta el build y nombra los culpables. No es un aviso.

Cuando necesites algo de Node, sepáralo: es exactamente por lo que `workspace.mjs` y `store.mjs` son navegables y `node-store.mjs` vive aparte.

3. Un año sin reglas debe fallar, nunca degradar

`loadRules(2027)` lanza si no existe `rules/2027.json`. Jamás lo hagas caer al año anterior.

Un cálculo con la tasa del año equivocado no falla: devuelve un número plausible que aparece meses después como una diferencia con el SII. `CONTRIBUTING.md` lo prohíbe por escrito y hay una prueba que lo fija.

4. Una tasa que se toca exige su fuente y su fecha

Cada cifra de `rules/<año>.json` lleva `source` (URL oficial) y `lastVerified`. Cambiar un número sin actualizarlos no es un descuido de estilo: rompe la trazabilidad, que es el argumento del producto.

Y después: `node scripts/build-rules.mjs`, o CI falla.

5. La documentación generada no se edita a mano

`docs/GLOSSARY.md`, `docs/EMPEZAR-AQUI.*` y `docs/ATAJOS-DE-TECLADO.md` se generan desde `packages/glossary`, `packages/onboarding` y `packages/shortcuts`. Editarlos directamente pone CI en rojo y tu cambio se perderá en la siguiente regeneración.

Edita el paquete y regenera.

Cómo está organizado el repositorio

```
packages/           El motor. Sin interfaz, sin node:*, es lo que viaja a las 3 plataformas
chile-tax-rules/    Las tasas por año, con su fuente
accounting-engine/  El cálculo: IVA, F29, CPT, patente, PPM, IDPC
company-operations/ La operación: CompanyWorkspace, capital, RUT, almacenes
glossary/ onboarding/ shortcuts/  Contenido, fuente de la documentación generada

apps/
web/               LA aplicación. 14 vistas. Lo demás son envoltorios
empresa-operativa/ Servidor local de archivos estáticos
android/           Envoltorio Capacitor (sin código propio)
contador-desktop/  Envoltorio Tauri + 4 comandos en Rust
contador-cli/      Línea de comandos

scripts/           13 scripts de build y verificación + 4 librerías compartidas
tests/             13 suites, 153 pruebas
docs/              Documentación de producto + esta serie
```

La forma más rápida de orientarse: casi todo lo importante está en dos archivos. `packages/company-operations/workspace.mjs` (1.388 líneas) tiene toda la lógica de negocio, y `packages/accounting-engine/index.mjs` todo el cálculo. Las cuatro superficies son envoltorios delgados sobre eso.

Itinerario de incorporación

Día 1 — entender el producto antes que el código

1. Usa la aplicación en **SANDBOX** durante media hora. Registra una venta, mira cómo cambia el borrador del F29, cierra un mes e intenta editar una operación de ese mes.

1. Lee `docs/EMPEZAR-AQUI.md`. Son 14 etapas y explica **para qué existe** el producto.

1. Lee [01 · Visión general](#), sobre todo «Qué NO hace».

2. Hojea `docs/GLOSSARY.md`. No hace falta memorizarlo, pero ****sí** entender por qué capital social, suscrito y enterado son tres cosas^{**}: es el corazón del producto.

Al final del día 1 deberías poder responder: ¿por qué la app se niega a marcar un trámite como hecho?

Día 2 — la arquitectura y sus reglas duras

1. `docs/ARCHITECTURE.md` — las siete decisiones de diseño.

⚠ Sus cifras de vistas y pruebas están desactualizadas ([H-03](#)); las decisiones, no.

1. [03 · Arquitectura](#) de esta serie: los diagramas y cómo se hacen cumplir las reglas.

1. Lee `scripts/build-web.mjs`, en concreto el gate `node:*`. Son quince líneas y explican media arquitectura.

1. Lee `packages/chile-tax-rules/index.mjs`. Son 1.599 bytes y contiene la regla más importante del sistema.

1. Ejecuta `node scripts/build-all.mjs` y mira qué produce.

Día 3 — el motor

1. `packages/accounting-engine/index.mjs` — empieza por `f29Basic`.

2. `tests/f29.test.mjs` en paralelo: las pruebas explican el comportamiento esperado mejor que cualquier comentario.

1. `packages/company-operations/workspace.mjs`, por secciones. Está dividido con comentarios; no lo leas de corrido.

1. [06 · Explicación profunda](#), flujos 1 y 2.

Ejercicio: sigue una venta desde el formulario hasta que aparece en el borrador del F29. Está descrito paso a paso en [08 · Flujo de datos](#); compruébalo tú.

Día 4 — la interfaz y las plataformas

1. `apps/web/src/app.js` (249 líneas) — el arranque y el ciclo de render.
2. `apps/web/src/lib/dom.js` — la plantilla `html`. **Entiende por qué escapa por defecto.**
3. `apps/web/src/lib/platform.js` — el único sitio donde se nota en qué plataforma corres.
4. Una vista sencilla, por ejemplo `views/panel.js`.
5. `apps/contador-desktop/src-tauri/src/lib.rs` — 135 líneas, los cuatro comandos.

Día 5 — cómo se publica y qué se verifica

1. `.github/workflows/ci.yml`. Lee los comentarios: explican por qué cada paso está ahí.
 2. `scripts/verify-apk.mjs`. **Es la pieza que mejor resume la cultura del proyecto.**
 3. [13 · Despliegue](#).
 4. [15 · Riesgos](#) — para saber dónde están las minas.
-

Dónde tocar cada cosa

Quiero...	Toca	Cuidado con
Cambiar una tasa o un plazo	<code>packages/chile-tax-rules/rules/<año>.json</code>	Actualizar <code>source</code> y <code>lastVerified</code> , y regenerar
Agregar un año tributario	Crear <code>rules/<año>.json</code> + regenerar	Verificar cada cifra contra la fuente oficial
Agregar un cálculo	<code>packages/accounting-engine/</code>	Que reciba el año y use <code>loadRules</code>
Agregar un tipo de operación	<code>TRANSACTION_KINDS</code> en <code>workspace.mjs</code>	Su <code>affectsVat</code> y su efecto en <code>periodSummary</code>
Agregar un trámite de constitución	<code>FORMATION_STEPS</code> en <code>workspace.mjs</code>	Aparecerá en las empresas existentes — es lo deseado
Agregar un campo a una entidad	<code>workspace.mjs</code> y <code>exportAll</code> / <code>importAll</code>	La compatibilidad con respaldos v1 y v2
Agregar una vista	<code>apps/web/src/views/</code> + registro en <code>app.js</code>	Nada de <code>node:*</code> . Escapa todo lo que venga del usuario
Agregar un término al glosario	<code>packages/glossary/</code>	No <code>docs/GLOSSARY.md</code> ; regenera
Agregar una etapa a la ruta	<code>packages/onboarding/</code>	Idem con <code>docs/EMPEZAR-AQUI.md</code>
Agregar un atajo	<code>packages/shortcuts/</code>	Idem con <code>docs/ATAJOS-DE-TECLADO.md</code>
Cambiar la persistencia	<code>store.mjs</code> / <code>node-store.mjs</code>	Respetar el contrato de seis métodos
Agregar un comando de escritorio	<code>src-tauri/src/lib.rs</code> + <code>invoke_handler</code>	Validar los argumentos ; no ampliar <code>capabilities</code> sin pensarlo
Cambiar la interfaz de la CLI	<code>apps/contador-cli/src/index.mjs</code>	Actualizar el texto de <code>AYUDA</code>
Tocar un workflow	<code>.github/workflows/</code>	Fijar cualquier acción nueva a un SHA de 40 caracteres

Cómo escribir una prueba

Sin framework, sin `describe`, sin *mocks*:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { CompanyWorkspace, createMemoryStore } from '../packages/company-operations/index.mjs';

test('descripción en español, en presente, describiendo la REGLA', () => {
  const ws = new CompanyWorkspace({ store: createMemoryStore(), mode: 'real' });
  // ...
  assert.throws(() => ws.addTransaction({ kind: 'sale', date: '2026-07-01' }), /cerrado/);
});
```

El estilo de los nombres importa. Fíjate en los que ya hay: «un período cerrado es inmutable en las dos direcciones», «reabrir un período exige motivo y queda en la bitácora». Describen la **regla de negocio**, no la función. Una prueba que se rompe debería decirle a quien la rompió qué garantía del producto acaba de perder.

`createMemoryStore()` da un espacio limpio en cada prueba: no hay fixtures que limpiar.

Convenciones que hay que respetar

Convención	Detalle
Idioma	Todo en español: comentarios, nombres de prueba, mensajes de error, documentación
Nombres de código	En inglés (<code>addTransaction</code> , <code>capitalEnterado</code> mezcla ambos por ser término legal chileno)
Comentarios	Explican por qué , no qué. Mira cualquier archivo: casi todos los comentarios justifican una decisión
Mensajes de error	En español y dirigidos a la persona, no al programador. Ejemplo: «Un paso sólo puede marcarse como realizado si registras su evidencia»
Módulos	ESM. Nada de <code>require</code>
Importes	Enteros en pesos. Se redondean con <code>money()</code>
Fechas	<code>YYYY-MM-DD</code> para días, <code>YYYY-MM</code> para períodos, ISO-8601 para marcas de tiempo
Sin dependencias	Antes de añadir una, comprueba si el repositorio ya resolvió el problema en <code>scripts/lib/</code>
Acciones de CI	Siempre fijadas a SHA, con la etiqueta como comentario
Commits	Mira <code>git log</code> : mensajes en español que explican la consecuencia , no el cambio

Sobre el último punto, un ejemplo real del historial: «fix: la presentación se publica como presentación, no como un `.html` en el repo». Ese es el registro.

Antes de abrir un pull request

```
pnpm check
```

Ejecuta los cinco gates de sincronía, la validación de reglas y las 153 pruebas. Es lo mismo que correrá CI.

Si tocaste la aplicación web:

```
node scripts/build-all.mjs
```

Si tocaste esta documentación:

```
node scripts/build-system-docs.mjs
```

Qué requiere especial cuidado

Zona	Por qué	Antes de tocar
<code>loadRules</code>	■ Es la única puerta a las tasas	Lee <code>CONTRIBUTING.md</code> y <code>tests/rules.test.mjs</code>
<code>f29Basic</code>	■ Un error produce una cifra plausible y equivocada	Lee <code>tests/f29.test.mjs</code> entero
<code>calculateTaxEquity</code>	■ El piso en cero y la elección de método son la ley, no una preferencia	<code>docs/tax/CAPITAL-PROPIO-TRIBUTARIO.md</code>
<code>calculateMunicipalPatent</code>	■ La base cambia según la etapa del negocio	<code>docs/municipal/PATENTE-MUNICIPAL.md</code>
<code>closeFiscalYear</code>	■ Congela una fotografía que debe seguir siendo válida en años	Entiende <code>legalRulesVersion</code>
El escapado en <code>dom.js</code>	■ Es el control anti-XSS del sistema	No introduces <code>raw()</code> con datos del usuario
<code>resolveSafe</code> en <code>server.mjs</code>	■ Traduce entrada de red a rutas de disco	El <code>+ path.sep</code> no es cosmético
<code>safe_mode</code> en <code>lib.rs</code>	■ Sin él, <code>mode</code> sería una escritura arbitraria	Las dos pruebas de al lado
Las claves del almacén	■ Cambiar una deja datos huérfanos	Piensa la migración primero
<code>identifier</code> de Tauri	■ Cambiarlo deja huérfanos los datos de escritorio	No lo cambies

Primeras tareas apropiadas para alguien junior

Ordenadas de menos a más. Todas salen de [15 · Riesgos](#), así que además de aprender, cierran deuda real.

#	Tarea	Qué se aprende	Hallazgo
1	Corregir «10 vistas» y «50 pruebas» en <code>docs/ARCHITECTURE.md</code>	Cómo se cuentan las cosas aquí	H-03
2	Borrar <code>apps/android/package-lock.json</code>	Por qué el gestor importa	H-04
3	Escribir las pruebas de <code>esc()</code> y <code>html</code>	El control de seguridad de la interfaz, con funciones puras	H-08
4	Añadir las tres claves que faltan en <code>FILES</code> de <code>node-store.mjs</code>	Cómo funciona el almacén de disco	H-05
5	Añadir un paso en CI que compare las dos <code>version</code>	Cómo se escribe un gate	H-10
6	Probar el saneo de <code>filename</code> en <code>lib.rs</code>	Rust, y el modelo de seguridad de escritorio	H-09
7	Capturar <code>QuotaExceededError</code> en <code>createWebStore</code> y avisar	El riesgo real de perder datos	H-01
8	Añadir la CSP como <code>meta</code> en <code>index.html</code>	Qué protege y qué no	H-02

Empieza por la 3. Es pequeña, es una función pura, y protege algo que hoy no tiene red.

Dónde preguntar

Duda	Documento
¿Por qué está hecho así?	<code>docs/ARCHITECTURE.md</code> → 03
¿Qué hace esta función?	05 · Referencia técnica
¿Cómo se encadena todo?	06 · Explicación profunda
¿Dónde se guarda esto?	07 · Persistencia
¿Qué significa esta palabra?	<code>docs/GLOSSARY.md</code> → 16
Me falla algo	14 · Troubleshooting
¿Qué está roto o pendiente?	15 · Riesgos
¿Qué viene después?	<code>docs/ROADMAP.md</code>
¿Cómo se opera la empresa?	<code>docs/RUNBOOK-MENSUAL.md</code> y <code>RUNBOOK-ANUAL.md</code>

Lo que no debes romper

`docs/ROADMAP.md` cierra con una sección titulada «**Reglas que ninguna versión romperá**». Léela antes de proponer un cambio grande. En resumen, y con lo verificado en esta revisión:

1. Las tasas nunca degradan a otro año.
 2. Un período cerrado no se modifica sin dejar rastro.
 3. La bitácora no se borra.
 4. EMPRESA REAL y SANDBOX no se mezclan.
 5. Nada se marca como hecho sin evidencia.
 6. Los datos no salen del dispositivo.
 7. Cuando la app y el SII no coincidan, manda el SII.
-